

VLSI COURSE PROJECT

CARRY LOOK-AHEAD ADDER

Sukeerthi Kattamuri

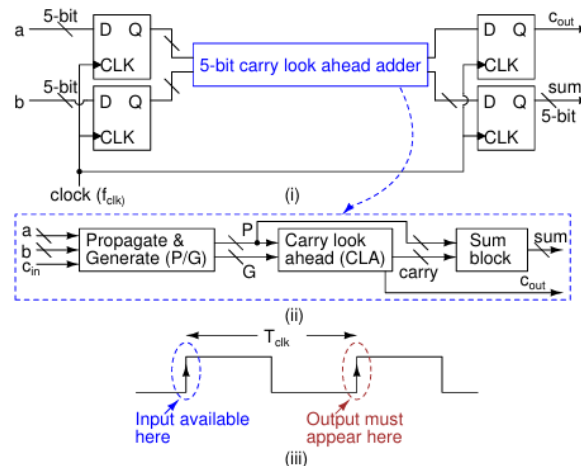
Roll No. 2024102074

I. INTRODUCTION

Addition is a fundamental operation required in digital circuits. One common approach for performing addition is the ripple-carry adder, where the carry output from each full adder is passed to the next stage. However, this introduces significant propagation delay because every stage must wait for the carry from the previous one.

To overcome this delay, a carry look-ahead adder (CLA) is used. In a CLA, carries are computed directly from the input bits using digital logic. This avoids the sequential delay seen in ripple adders and results in much faster computation.

The basic circuit diagram is given below



CLA uses two signals for each bit position:

Propagate: $P_i = A_i \oplus B_i$

Generate: $G_i = A_i \cdot B_i$

Using these signals, the carry-out of each stage is expressed as: $C_{i+1} = G_i + P_i \cdot C_i$. We can recursively calculate the carry bit for each bit using this formula as done below.

$$C_1 = g_1 + p_1 \cdot C_0$$

$$C_2 = g_2 + p_2 \cdot C_1 = g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot C_0$$

$$C_3 = g_3 + p_3 \cdot C_2 = g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot C_0$$

$$C_4 = g_4 + p_4 \cdot C_3 = g_4 + p_4 \cdot g_3 + p_4 \cdot p_3 \cdot g_2 + p_4 \cdot p_3 \cdot p_2 \cdot g_1 + p_4 \cdot p_3 \cdot p_2 \cdot p_1 \cdot C_0$$

$$C_5 = g_5 + p_5 \cdot C_4 = g_5 + p_5 \cdot g_4 + p_5 \cdot p_4 \cdot g_3 + p_5 \cdot p_4 \cdot p_3 \cdot g_2 + p_5 \cdot p_4 \cdot p_3 \cdot p_2 \cdot g_1 + p_5 \cdot p_4 \cdot p_3 \cdot p_2 \cdot p_1 \cdot C_0$$

Once the carries are computed, we can calculate all the sum bits.

$$S_i = P_i \oplus C_{i-1}$$

IIA. CLA DESIGN TOPOLOGY

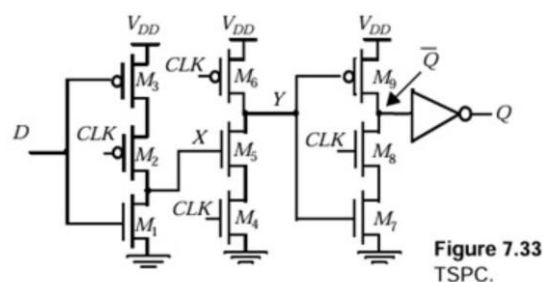
Here we use static CMOS implementation to make the required logic gates. We require multiple input gates for performing these computations. This can either be implemented by using a single multiple input gate or by using multiple dual input gates. After analysis of ngSpice simulation, there was less delay from using dual input gates. Hence all the required gates AND(input 2, 3, 4, 5, 6), OR(input 2, 3, 4, 5, 6), NOT and XOR were implemented using static CMOS logic.

The reason for preferring this logic style over other dynamic styles is that we require for the output to be computed within one clock cycle, this is difficult to achieve through dynamic implementations. Dynamic logic also has the risk of charge leakage. Also dynamic logic requires a precharge and evaluation phase which lead to excessive power consumption that is not preferred.

Other advantages of the CMOS style are its simple and straightforward design. Moreover it has high noise margins, giving a more robust implementation.

The basic circuit that we used can be divided into three blocks. The first block consists of flip-flops. The implementation used here is a TSPC (True Single Phase Clock) flip-flop. The flip-flop ensures that the input bits are all given simultaneously to the adder to prevent any glitches. In the next stage, the logic signals discussed above are implemented using static CMOS designed gates. The carry bits are all computed simultaneously without needing to wait for previous computations. Once the sum bits and final carry bit is generated, they are stored in another set of flip-flops, so that the results can be used at the next rising clock edge.

The circuit implemented for the TSPC flip-flop is given below.



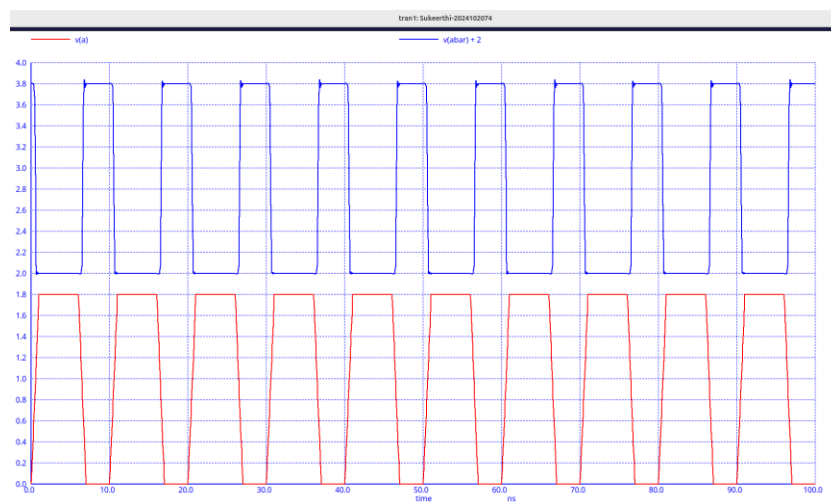
II.B CLA TRANSISTOR SIZING

Minimum sizing MOSFETS are used. $\lambda = 0.09 \mu\text{m}$ is taken. We are required that for the NOT gate $W_P / W_N = 2$.

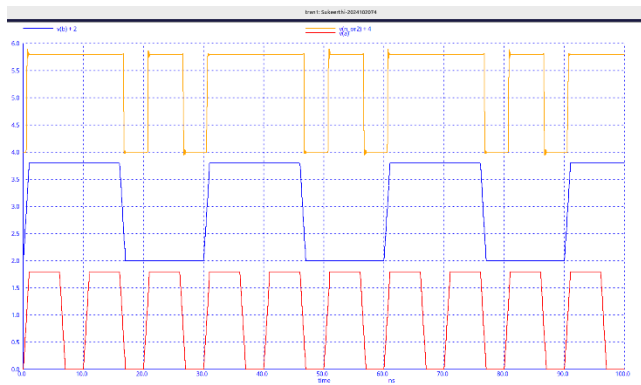
OR gates are implemented by using a NOR gate followed by an inverter and similarly AND gates are implemented using a NAND gate followed by an inverter.

Gate	NMOS	PMOS
NOT	900 nm	1800 nm
NOR – 2 input	900 nm	3600 nm
NOR – 3 input	900 nm	5400 nm
NOR – 4 input	900 nm	7200 nm
NOR – 5 input	900 nm	9000 nm
NOR – 6 input	900 nm	10800 nm
NAND – 2 input	1800 nm	1800 nm
NAND – 3 input	2700 nm	1800 nm
NAND – 4 input	3600 nm	1800 nm
NAND – 5 input	4500 nm	1800 nm
NAND – 6 input	5400 nm	1800 nm
XOR – 2 input	1800 nm	3600 nm
TSPC Flip Flop	900 nm	1800 nm

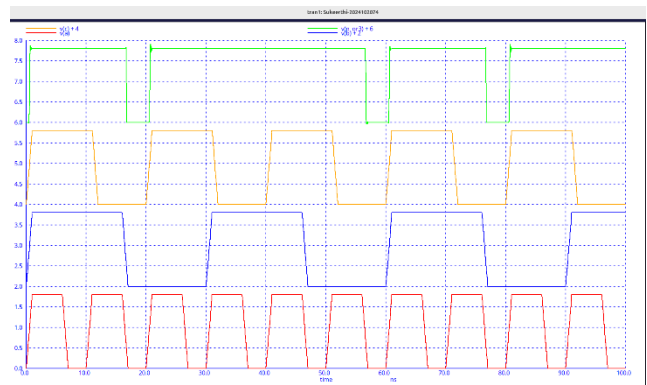
III. NSPICE SIMULATIONS



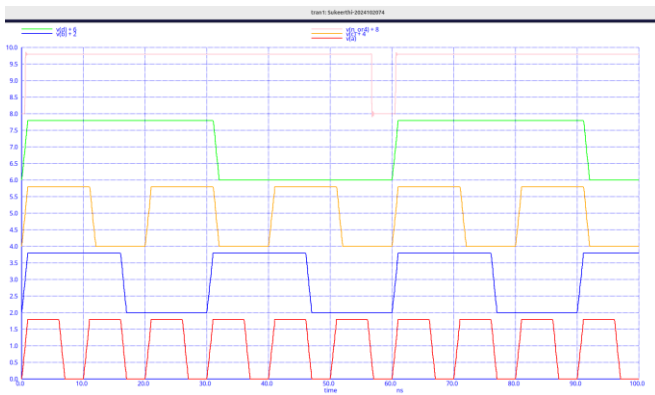
Inverter



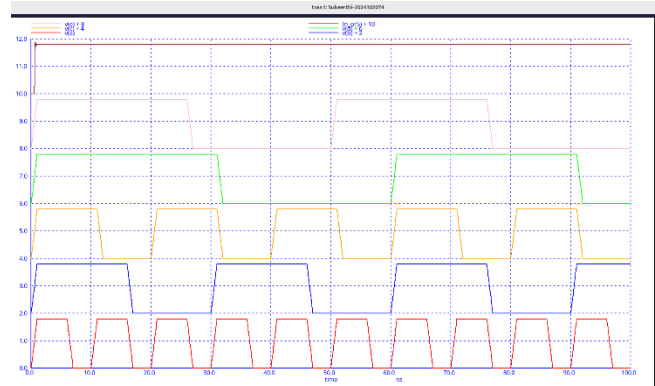
OR – 2 input



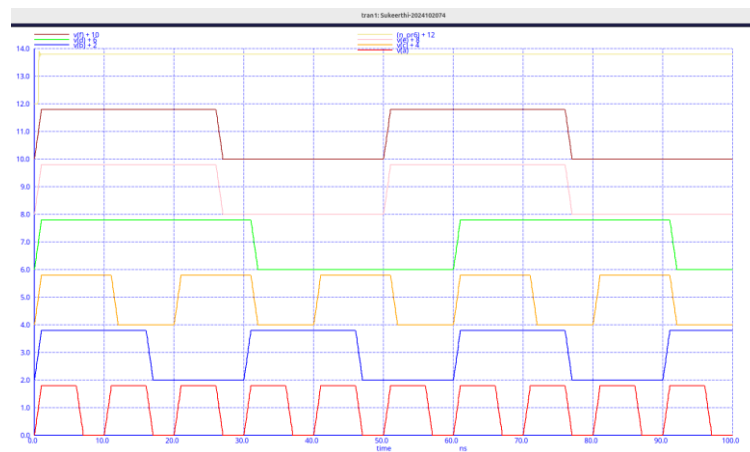
OR – 3 input



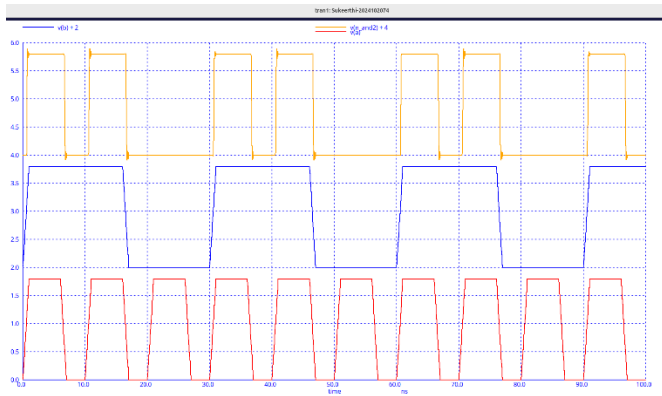
OR – 4 input



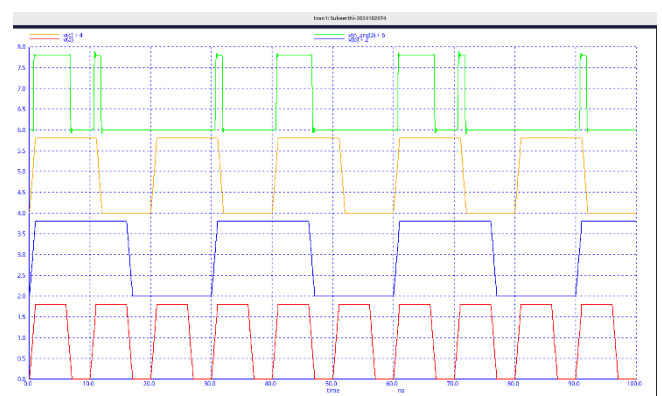
OR – 5 input



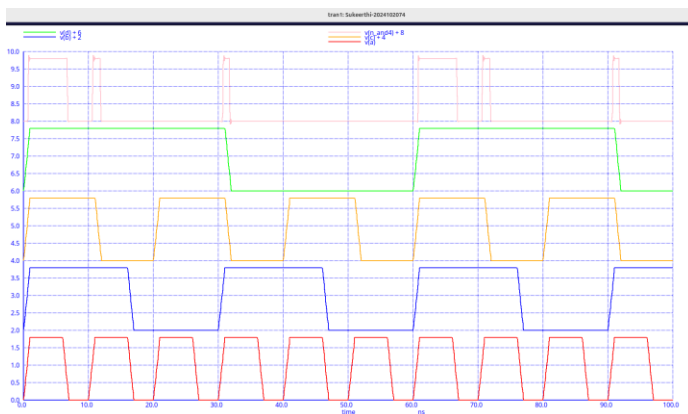
OR – 6 input



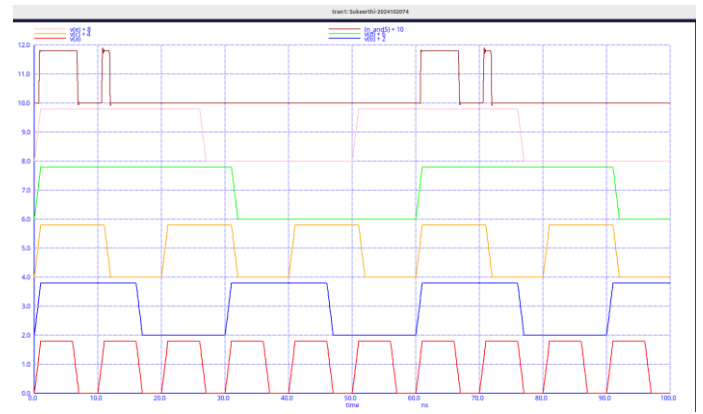
AND - 2 input



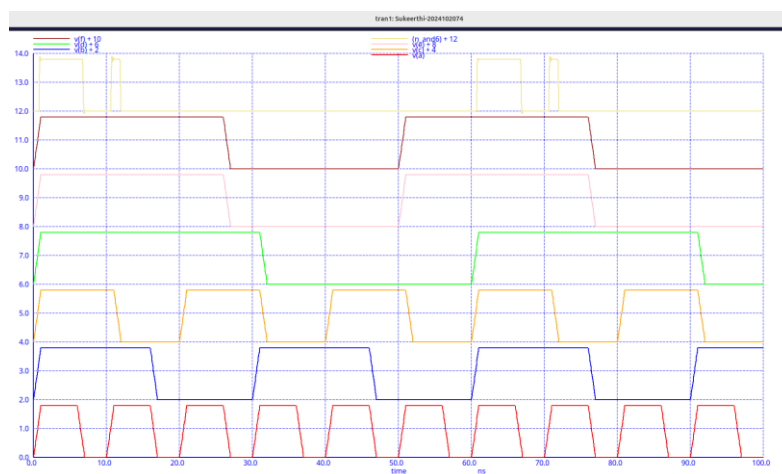
AND – 3 input



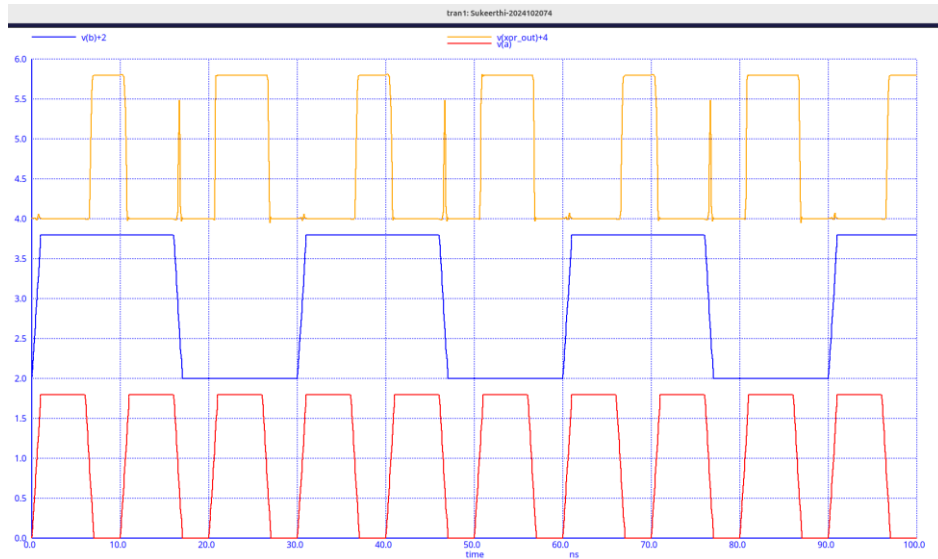
AND – 4 input



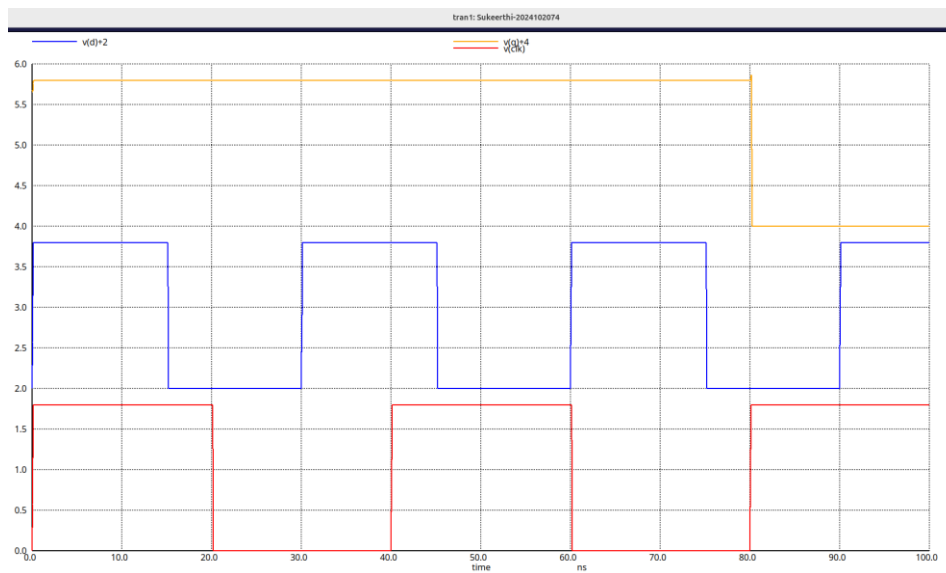
AND – 5 input



AND – 6 input



XOR – 2 input



Positive Edge Triggered TSPC Flip Flop

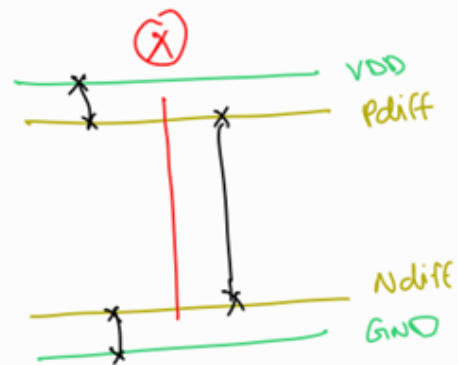
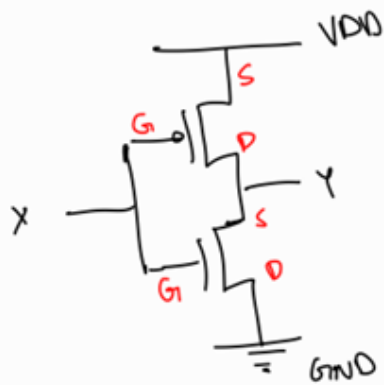
IV. TSPC FLIP FLOP SETUP TIME AND HOLD TIME

```

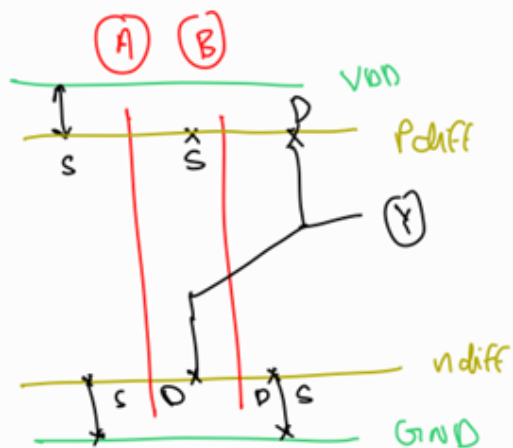
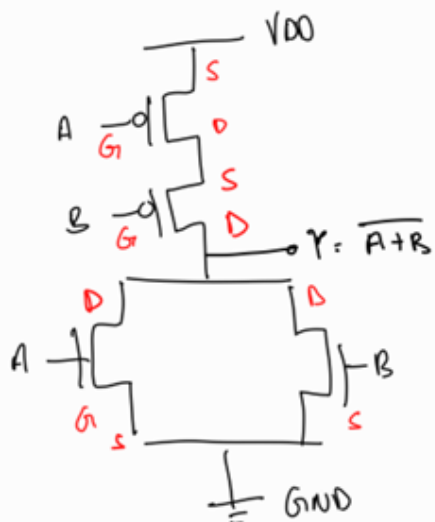
setup      = 4.528879e-11
hold       = 4.007412e-08
  
```

V. STICK DIAGRAMS FOR IMPLEMENTATION

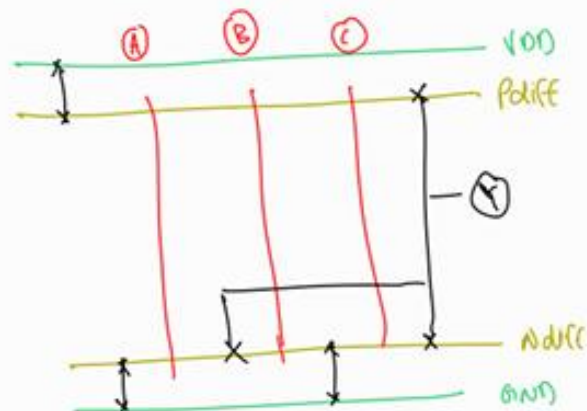
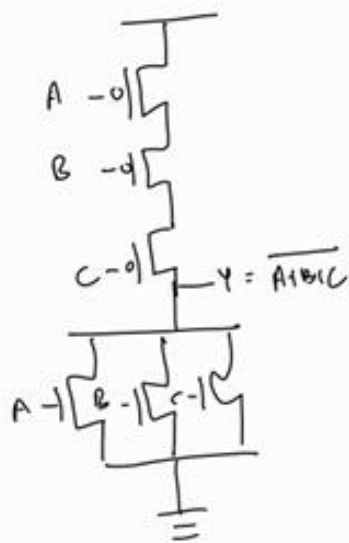
CMOS inverter



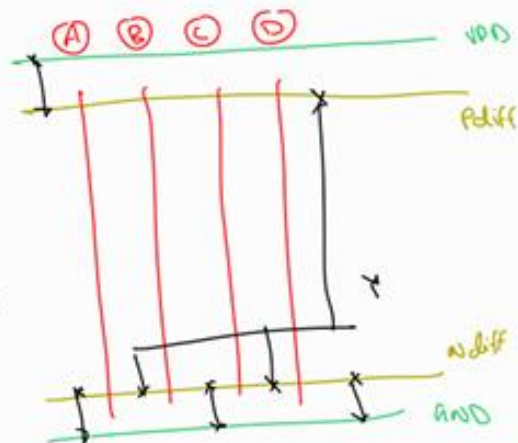
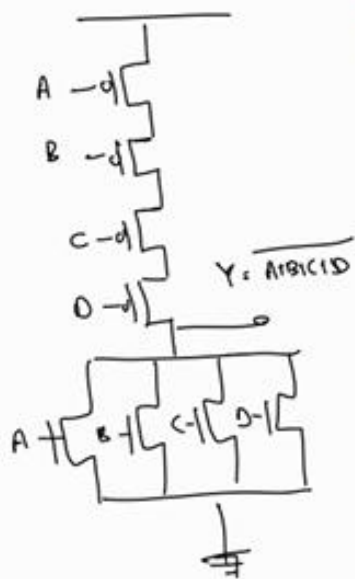
2 input NOR



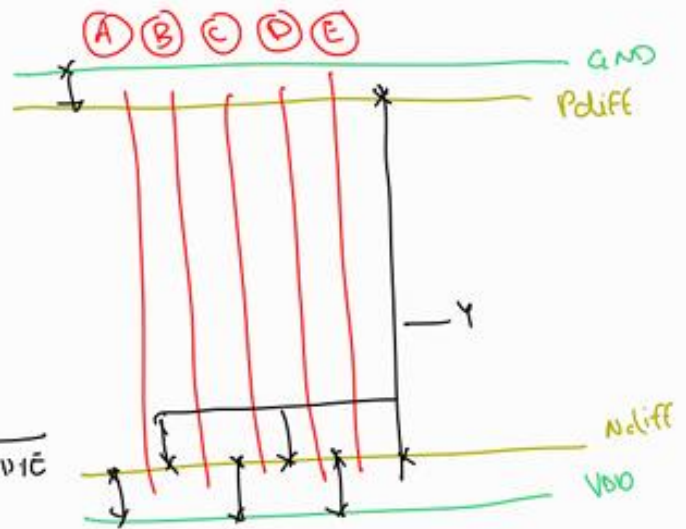
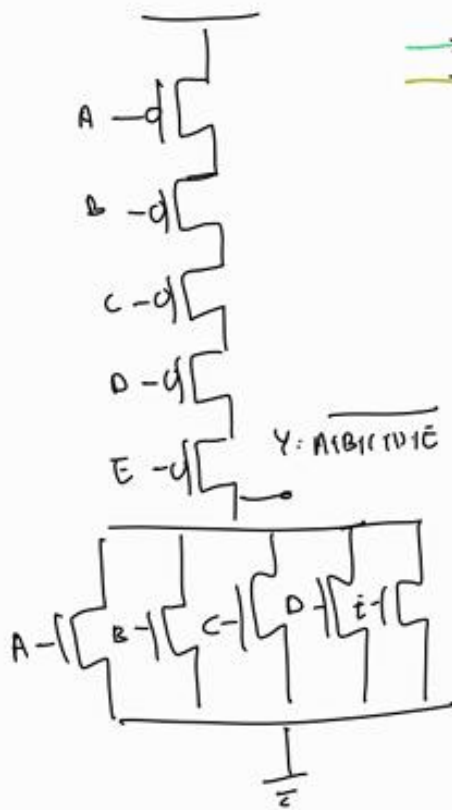
3 Input NOR



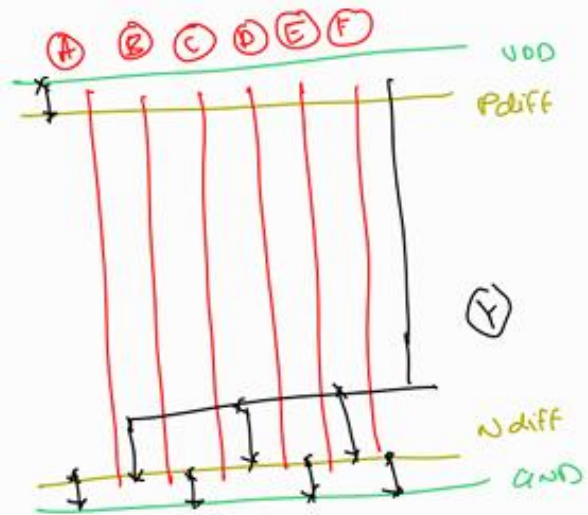
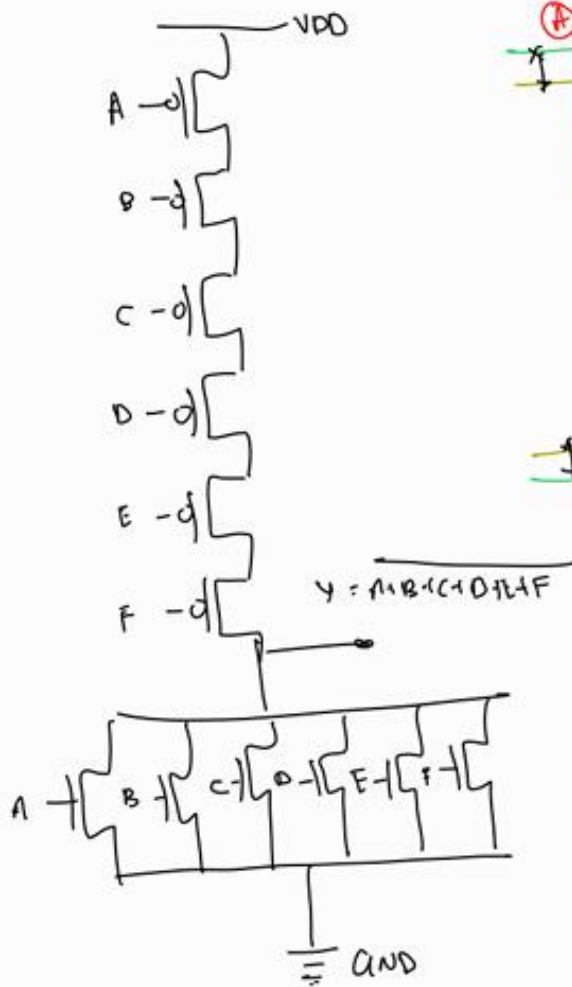
4 Input NOR



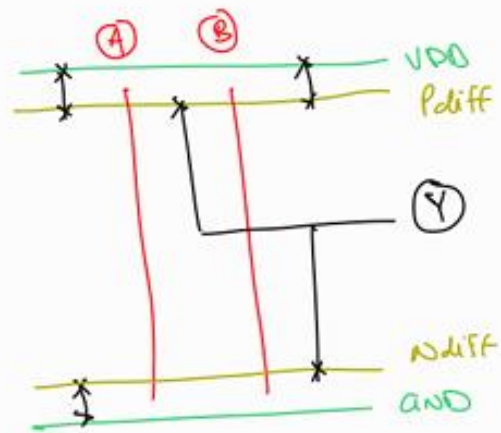
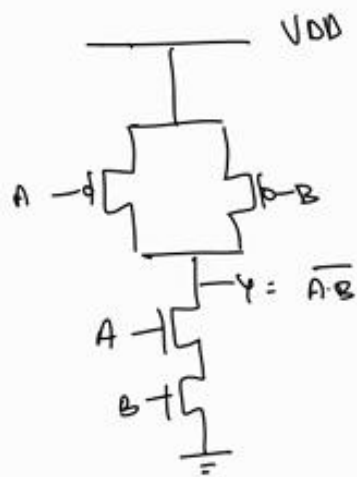
5 Input NOR



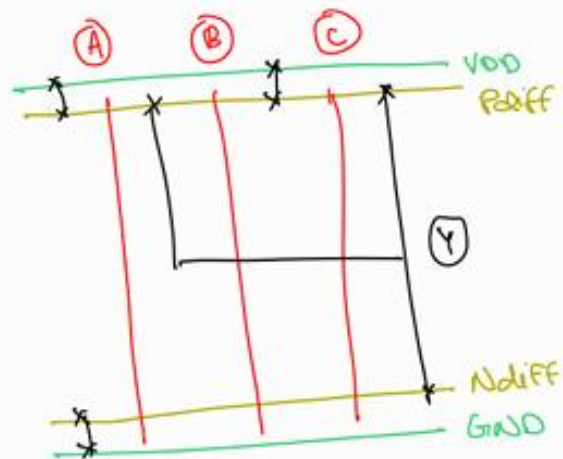
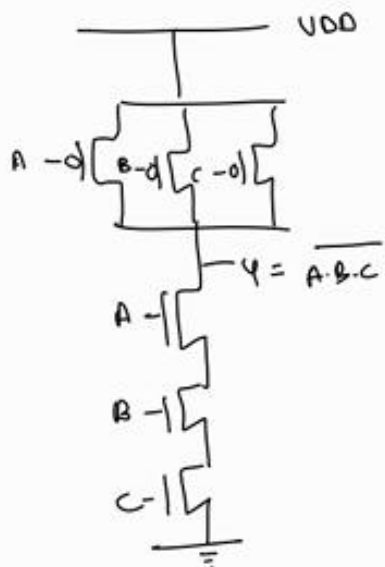
6 input NOR

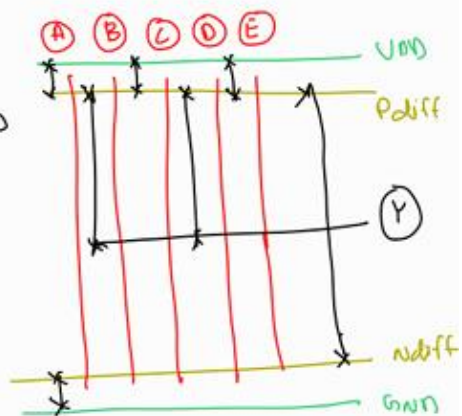
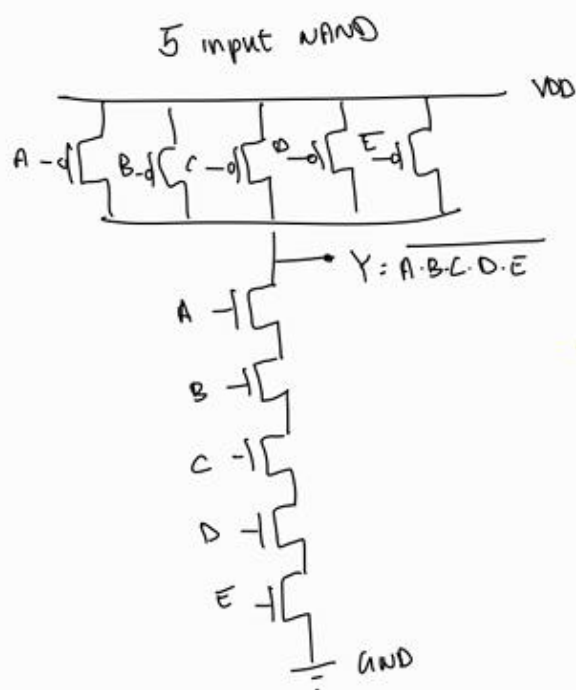
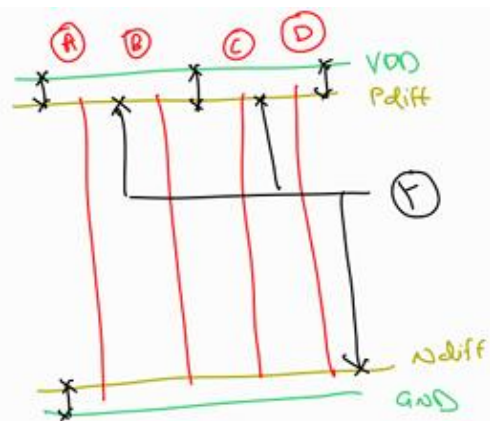
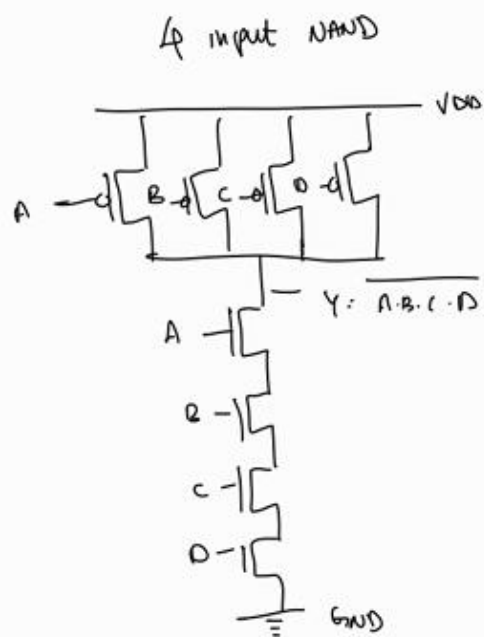


2 input NAND

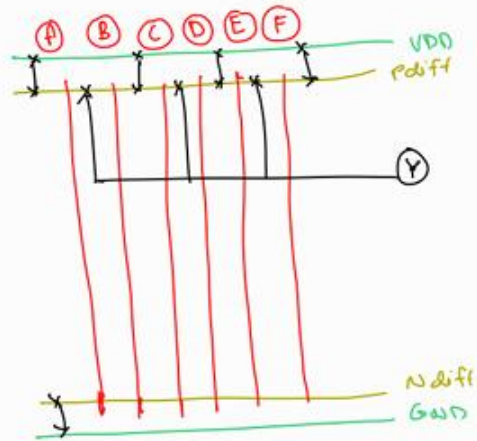
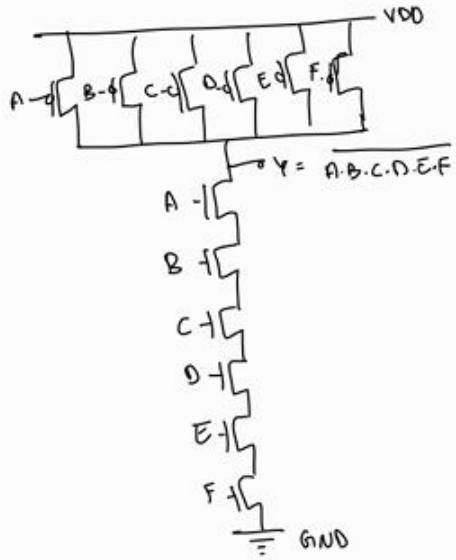


3 input NAND

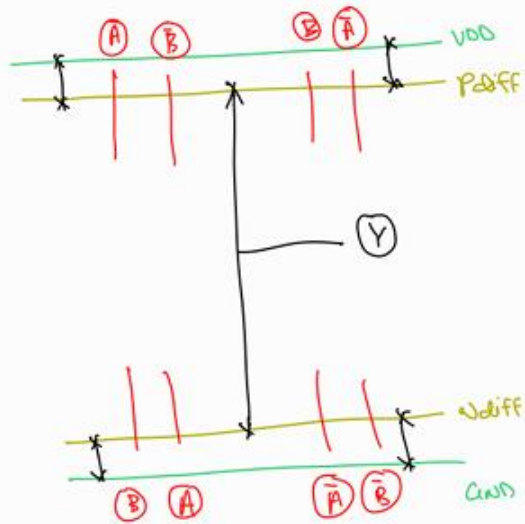
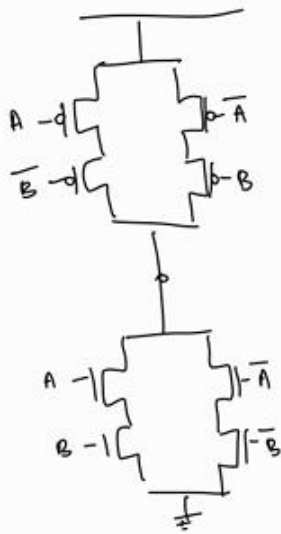




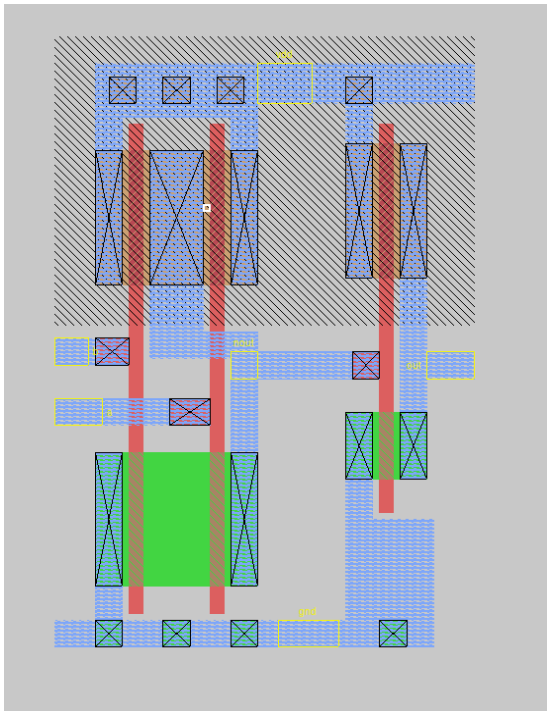
6 input NAND



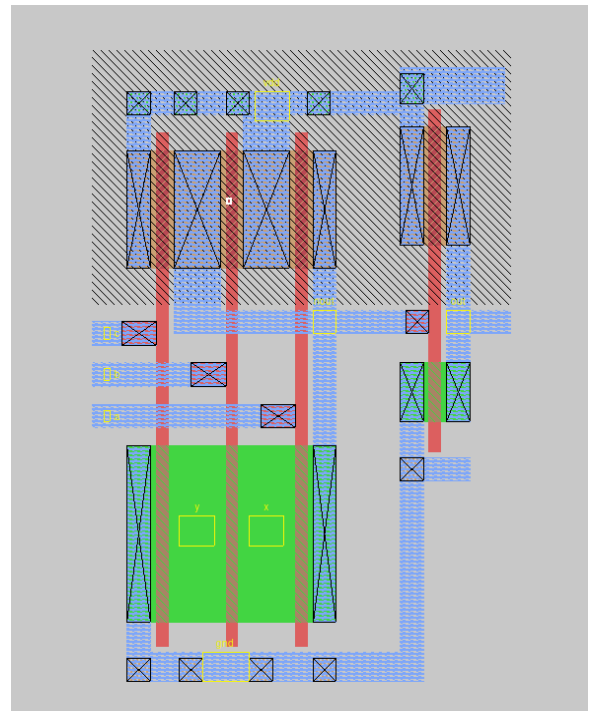
2 input XOR



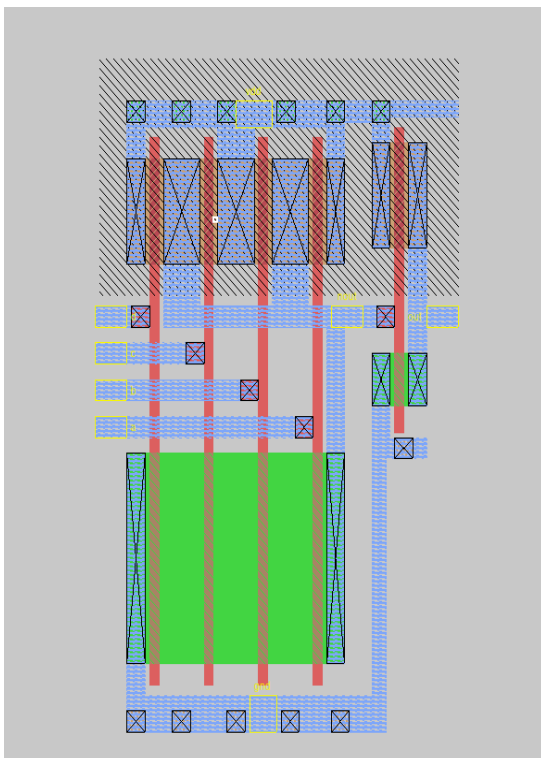
VI. LAYOUT MADE IN MAGIC



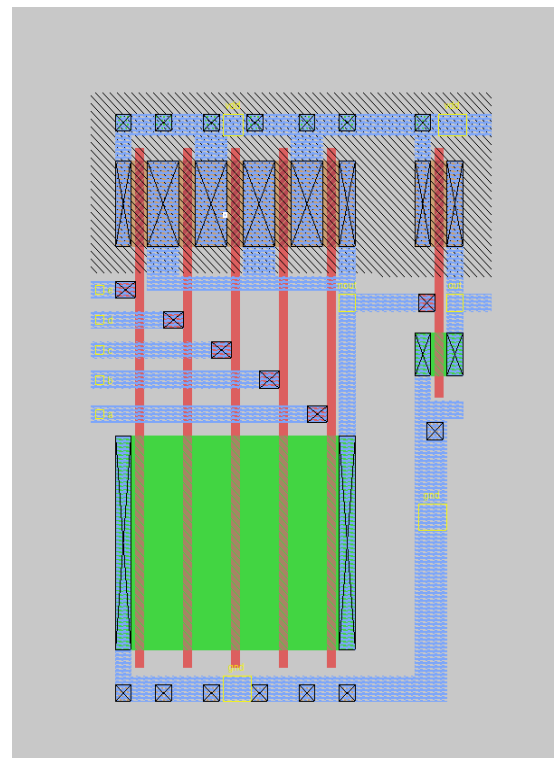
AND – 2 input



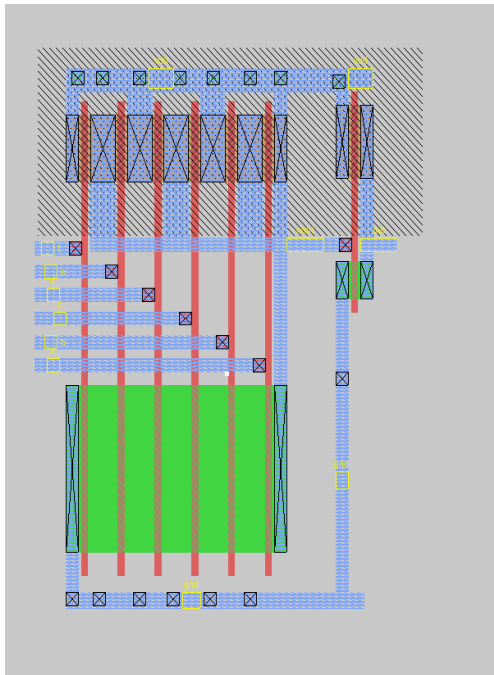
AND – 3 input



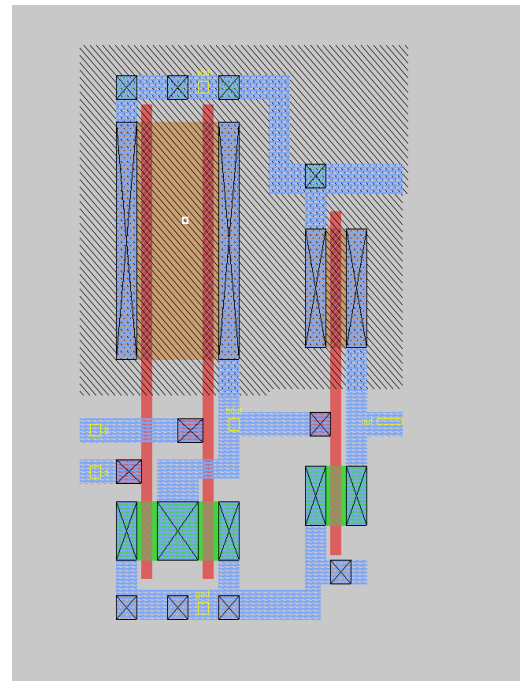
AND – 4 input



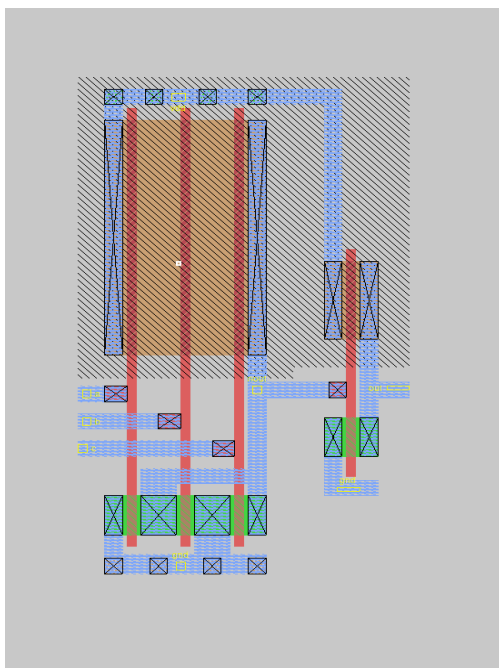
AND – 5 input



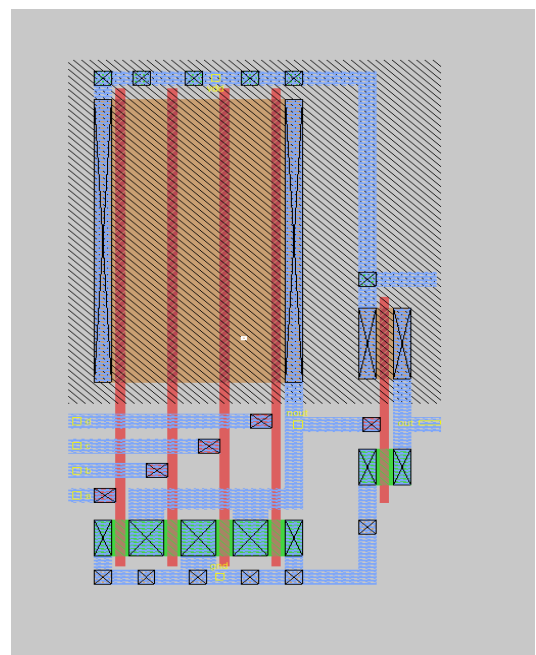
AND – 6 Input



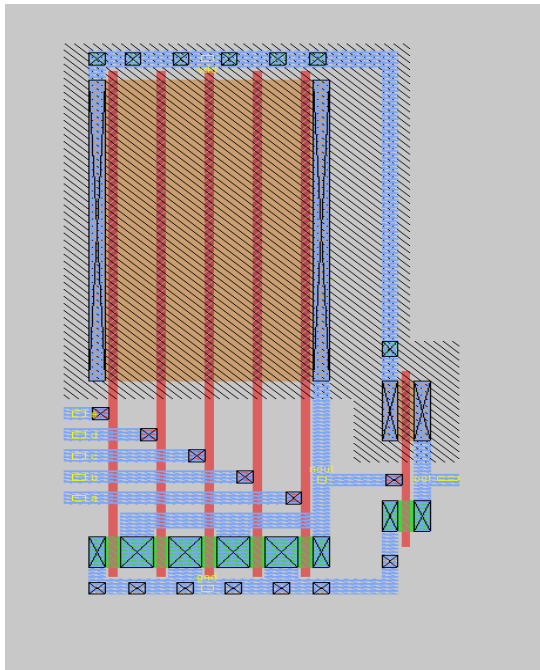
OR – 2 input



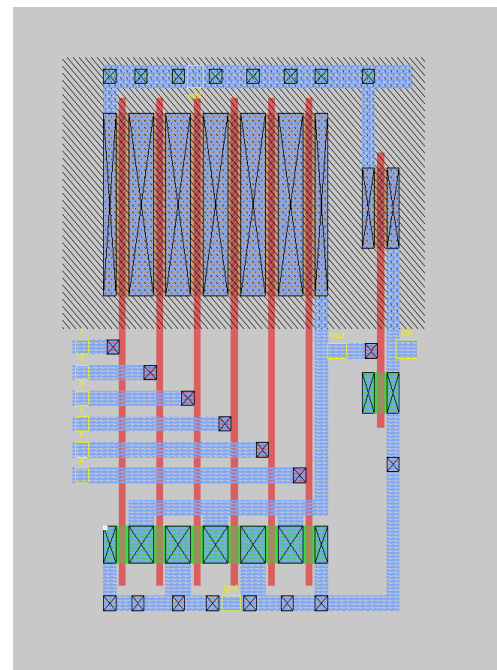
OR – 3 input



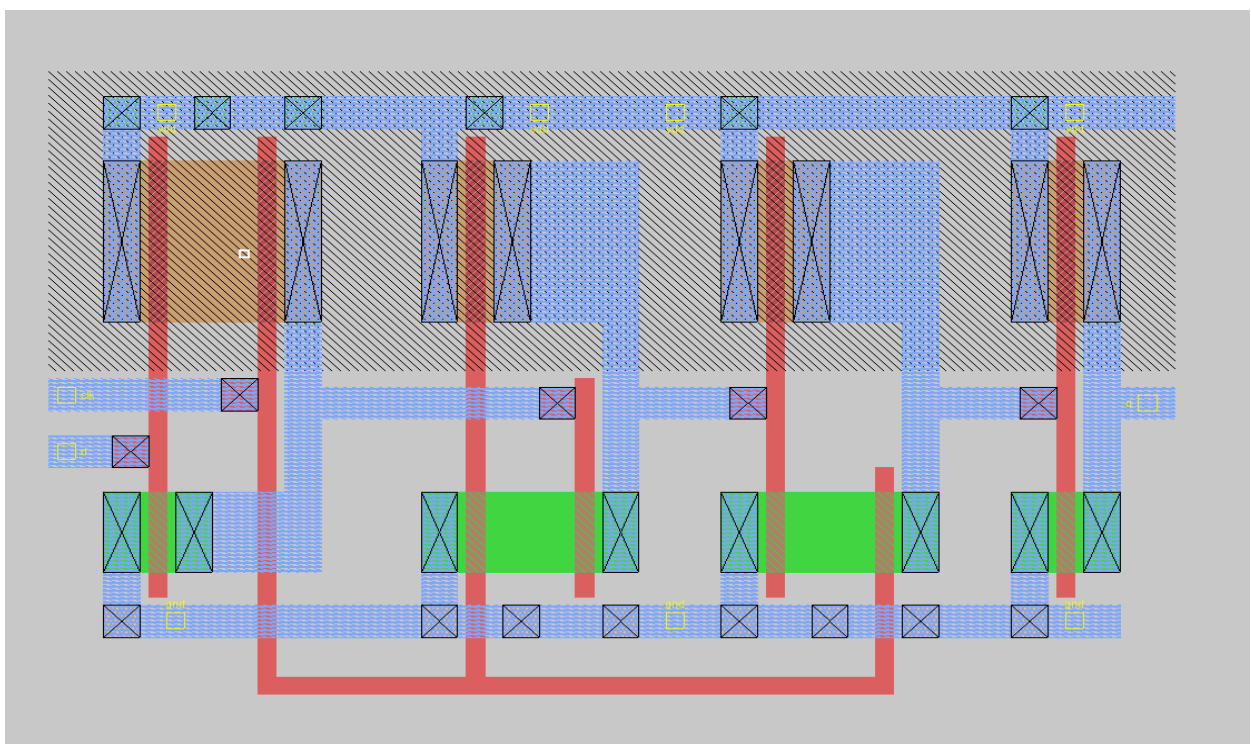
OR – 4 input



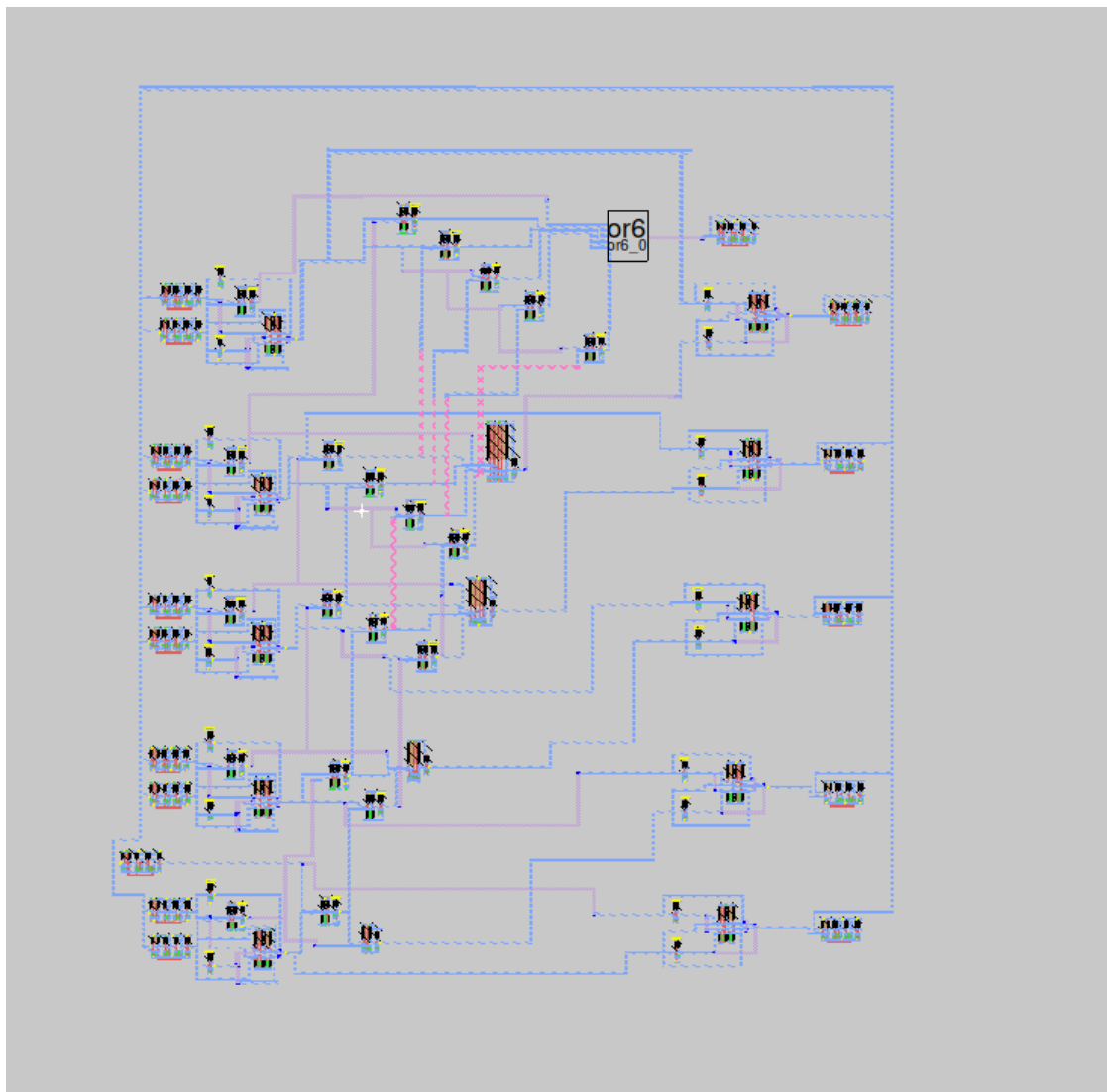
OR – 5 input



OR – 6 Input

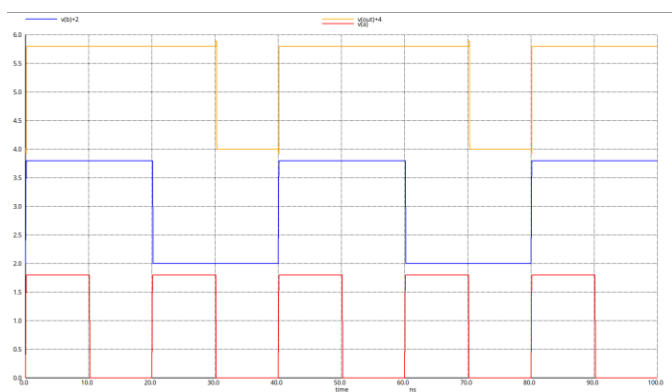


TSPC Flip Flop

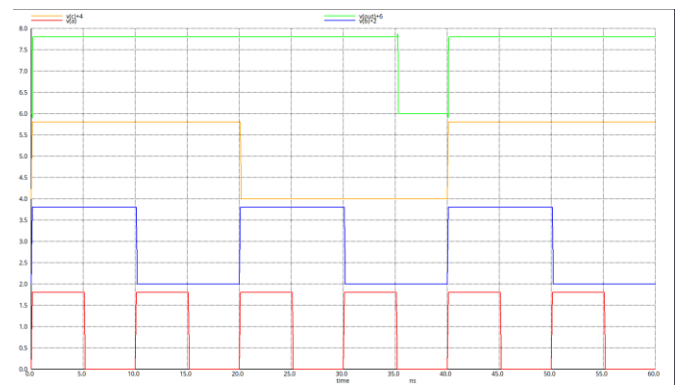


Final Implementation

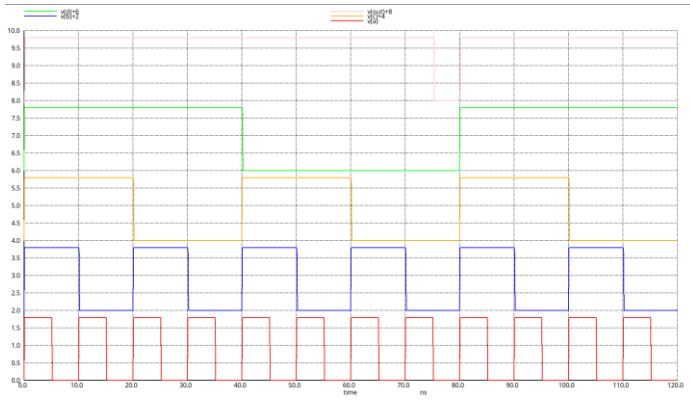
VII. POST LAYOUT SIMULATIONS



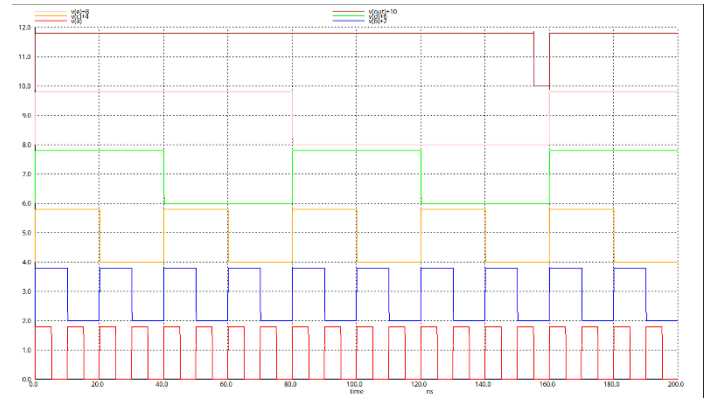
OR – 2 input



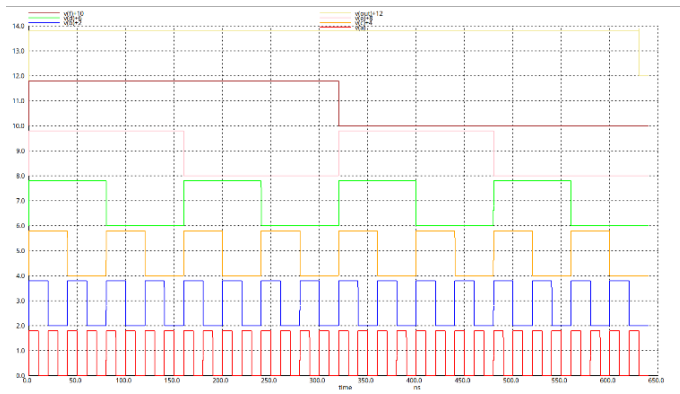
OR – 3 input



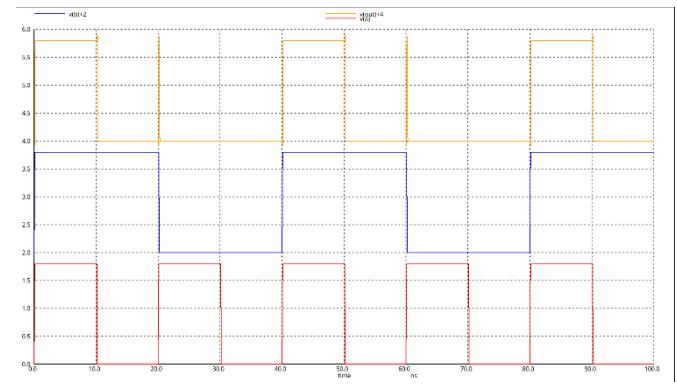
OR – 4 input



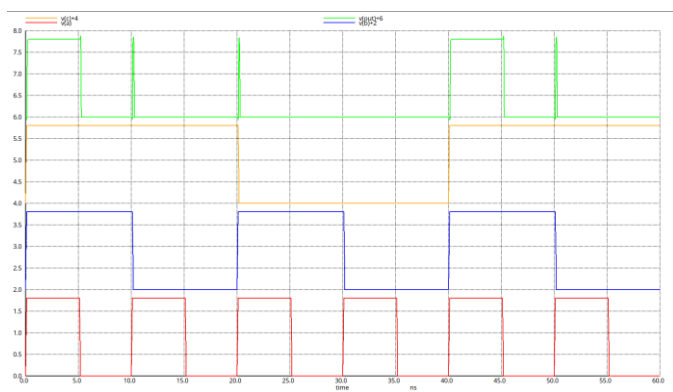
OR – 5 input



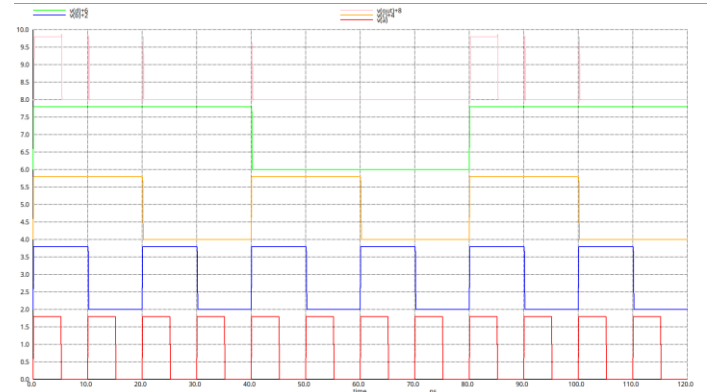
OR – 6 input



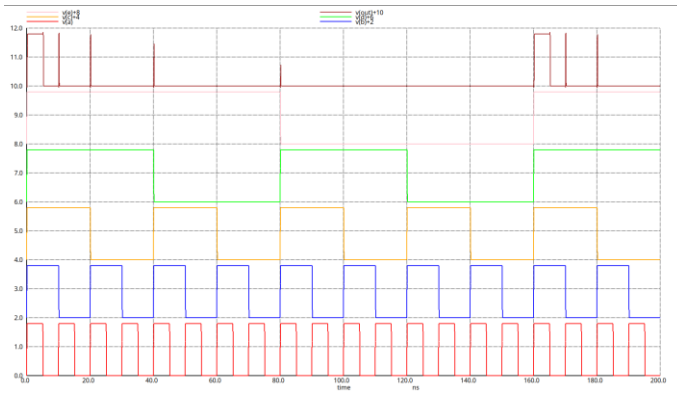
AND – 2 input



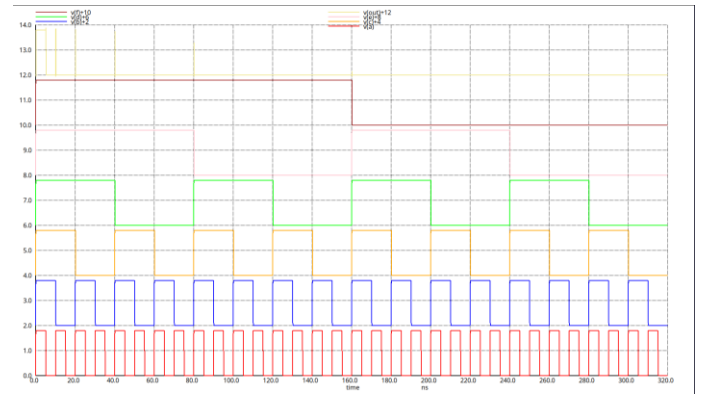
AND – 3 input



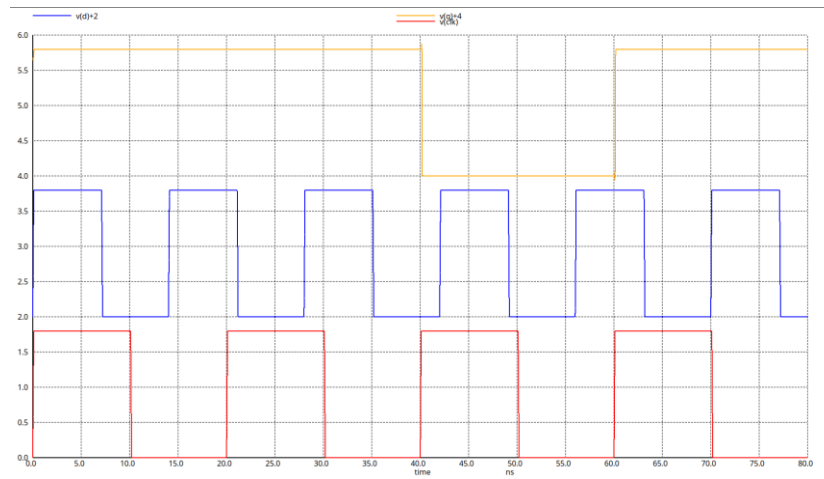
AND – 4 input



AND – 5 input



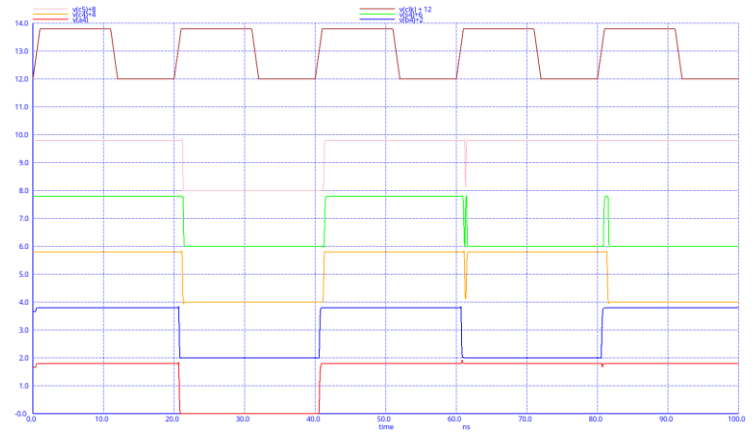
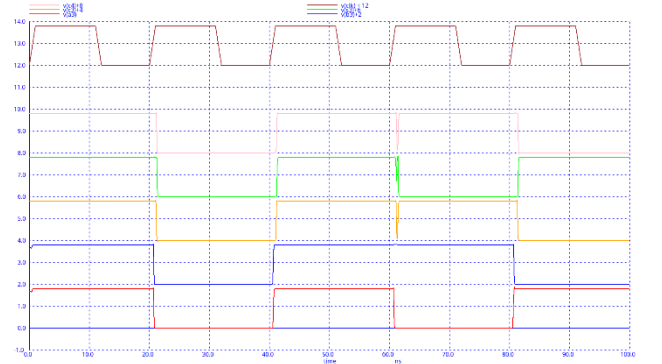
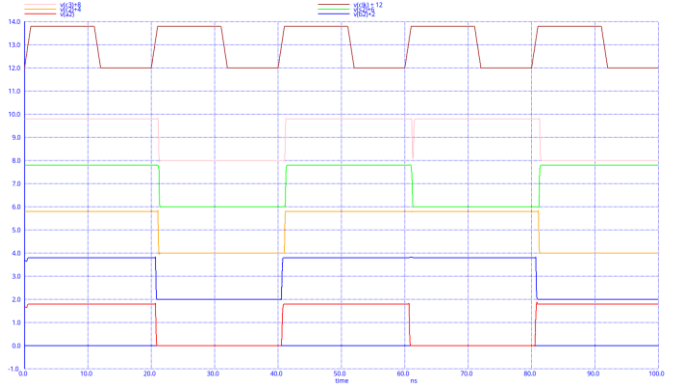
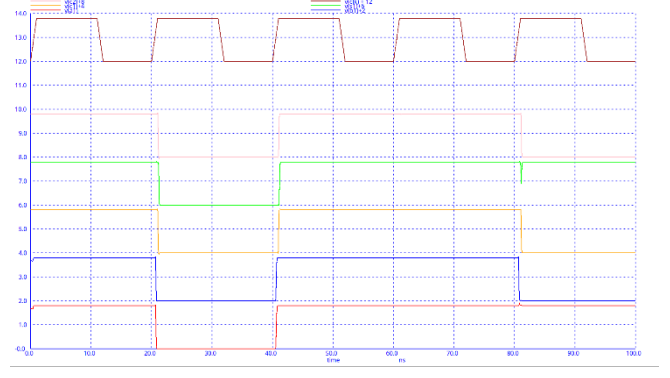
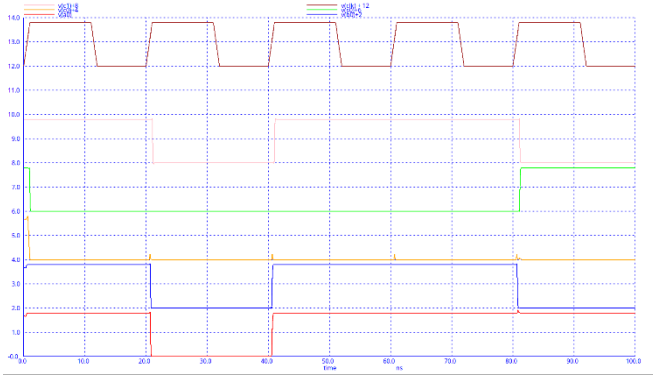
AND – 6 input



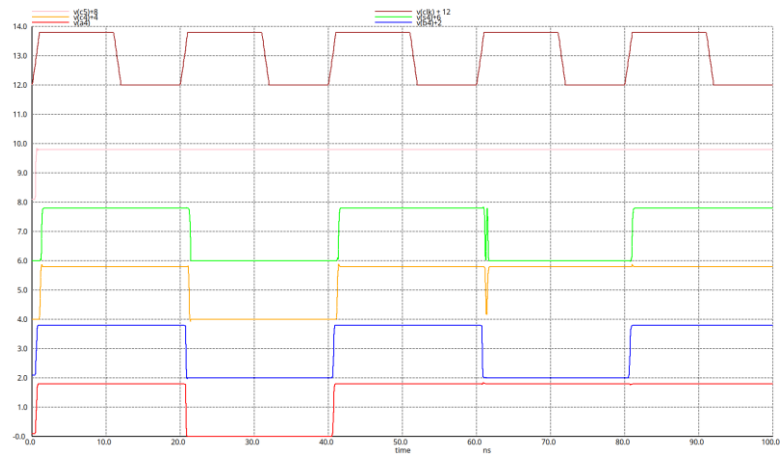
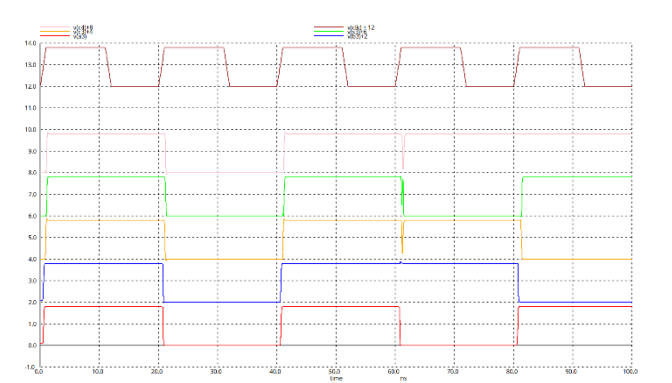
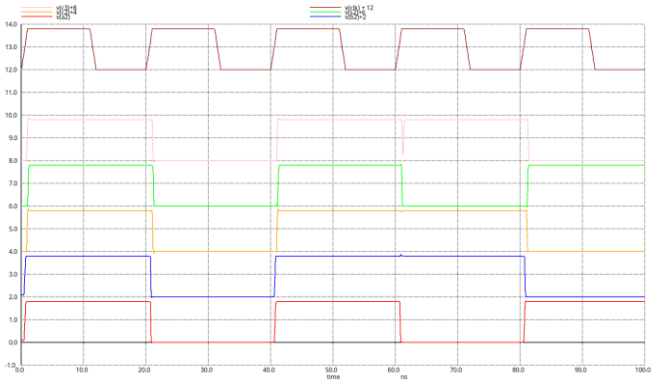
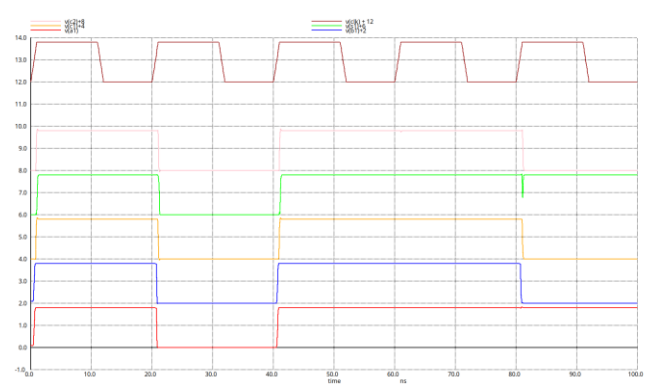
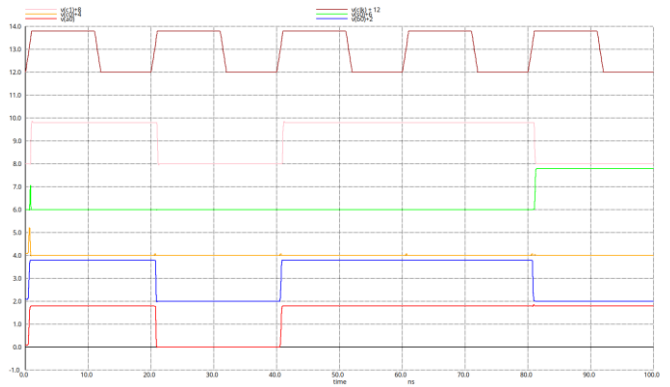
TSPC Flip Flop Output

IX. FULL CIRCUIT

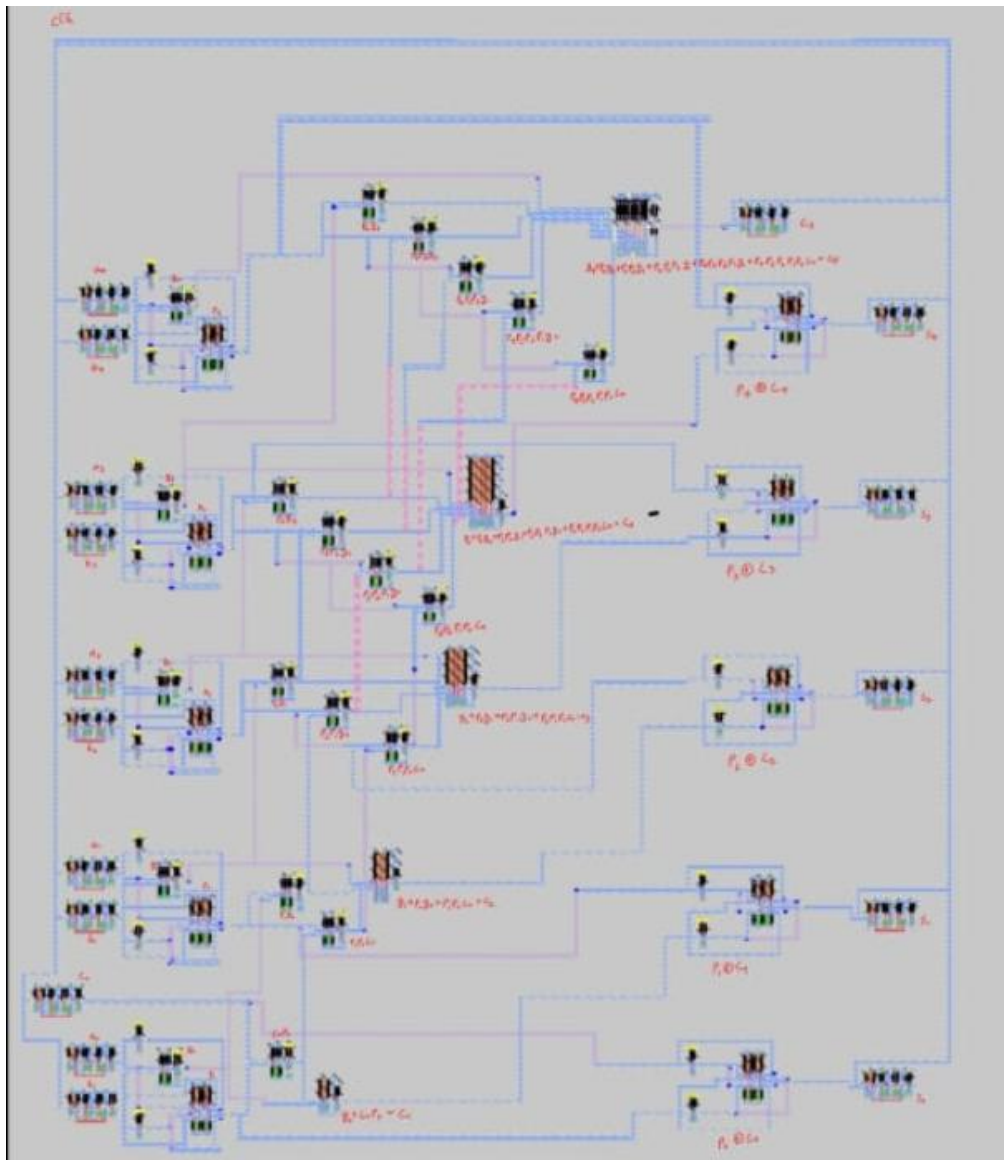
A. PRE-LAYOUT



B. POST LAYOUT

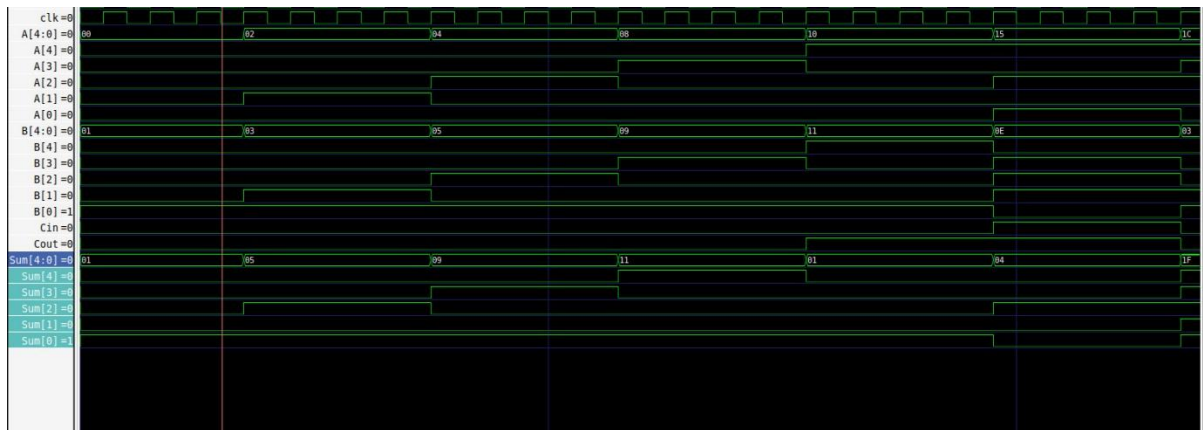


VIII. FLOOR PLAN



XI. VERILOG

The GTK wave from the Verilog code is given below:



XII. FPGA Implementation

